

AICE 샘플 문제 학습 가이드

데이터셋 설명

A0007IT.json

- **Time_Departure**: 출발시각
- **Time_Arrival**: 도착시각
- **Distance**: 이동 거리 (m)
- **Speed_Per_Hour**: 평균시속
- **Address1**: 주소1
- **Address2**: 주소2
- **Weekday**: 출발시각의 요일
- **Hour**: 출발시각의 시각
- **Day**: 출발시각의 날짜

signal.csv

- **RID**: 경로ID
- **Signaltype**: 경로의 신호등 갯수

문제 1: 라이브러리 임포트

문제: scikit-learn을 별칭 sk로 임포트하세요.

답안:

```
python  
  
import sklearn as sk
```

핵심 개념:

- 라이브러리 별칭(alias) 사용으로 코드 간소화
- scikit-learn은 머신러닝의 대표적인 파이썬 라이브러리

문제 2: Pandas 임포트

문제: Pandas를 별칭 pd로 임포트하세요.

답안:

```
python
```

```
import pandas as pd
```

핵심 개념:

- pd는 Pandas의 표준 별칭
- 데이터 분석의 핵심 라이브러리

문제 3: 데이터 읽기 및 병합

문제: 두 데이터 파일을 읽고 RID 기준으로 inner join하여 df에 저장하세요.

답안:

```
python
```

```
# 데이터 파일 읽기
```

```
df_a = pd.read_json('A0007IT.json')
```

```
df_b = pd.read_csv('signal.csv')
```

```
# 데이터 병합
```

```
df = pd.merge(df_a, df_b, on='RID', how='inner')
```

핵심 개념:

- `read_json()`: JSON 파일 읽기
- `read_csv()`: CSV 파일 읽기
- `merge()`: 데이터프레임 병합
- `on='RID'`: 병합 기준 컬럼
- `how='inner'`: 교집합만 유지

문제 4: 데이터 시각화 (CountPlot)

문제: Address1의 분포를 countplot으로 표시하고, '-' 값을 가진 행을 삭제하세요.

답안:

```
python
```

```

import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
import seaborn as sns

plt.rc('font', family='NanumGothicCoding')

# CountPlot 생성
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='Address1')
plt.xticks(rotation=45)
plt.show()

# '-' 값을 가진 행 삭제
df = df[df['Address1'] != '-']

# 답안 (옳지 않은 선택지)
답안04 = 3 # 지역명이 없는 '-'에 해당되는 row가 2개 있다는 것이 틀림

```

핵심 개념:

- `countplot()`: 범주형 데이터의 빈도 시각화
- 한글 폰트 설정 필요
- 결측값 처리의 중요성

문제 5: 관계 시각화 (JointPlot)

문제: Time_Driving과 Speed_Per_Hour의 관계를 jointplot으로 표시하세요.

답안:

```

python

# Time_Driving 계산 (도착시간 - 출발시간)
df['Time_Driving'] = pd.to_datetime(df['Time_Arrival']) - pd.to_datetime(df['Time_Departure'])
df['Time_Driving'] = df['Time_Driving'].dt.total_seconds() / 60 # 분 단위 변환

# JointPlot 생성
sns.jointplot(data=df, x='Time_Driving', y='Speed_Per_Hour', kind='scatter')
plt.show()

```

핵심 개념:

- `jointplot()`: 두 변수 간의 관계와 각각의 분포를 동시에 시각화
- 시간 데이터 처리 및 계산

- 이상치 탐지의 중요성

문제 6: 이상치 제거 및 전처리

문제: 시속 300 이상의 이상치를 제거하고 RID 컬럼을 삭제하세요.

답안:

```
python

# 시속 300 이상 이상치 제거
df_temp = df[df['Speed_Per_Hour'] < 300].copy()

# RID 컬럼 삭제
df_temp = df_temp.drop('RID', axis=1)
```

핵심 개념:

- 이상치(outlier) 탐지 및 제거
- `drop()`: 컬럼 삭제
- `copy()`: 데이터프레임 복사로 원본 보호

문제 7: 결측치 처리

문제: 결측치를 확인하고 제거한 후, 결측치 개수를 답안07에 저장하세요.

답안:

```
python

# 결측치 확인
print(df_temp.isnull().sum())

# 결측치 개수 계산
답안07 = df_temp.isnull().sum().sum()

# 결측치가 있는 행 삭제
df_na = df_temp.dropna()

print(f"결측치 개수: {답안07}")
```

핵심 개념:

- `isnull()`: 결측치 확인
- `dropna()`: 결측치가 있는 행 제거

- 데이터 품질 관리의 중요성
-

문제 8: 불필요한 컬럼 삭제

문제: Time_Departure, Time_Arrival 컬럼을 삭제하세요.

답안:

```
python

# 불필요한 컬럼 삭제
df_del = df_na.drop(['Time_Departure', 'Time_Arrival'], axis=1)
```

핵심 개념:

- 여러 컬럼 동시 삭제
 - 모델링에 불필요한 변수 제거
 - 차원 축소(dimensionality reduction)
-

문제 9: 원-핫 인코딩

문제: object 타입 컬럼들을 원-핫 인코딩하세요.

답안:

```
python

# object 타입 컬럼 확인
print(df_del.dtypes)

# 원-핫 인코딩
df_preset = pd.get_dummies(df_del, dtype=int)
```

핵심 개념:

- 원-핫 인코딩: 범주형 데이터를 이진 벡터로 변환
 - `get_dummies()`: 판다스의 원-핫 인코딩 함수
 - 머신러닝 모델을 위한 데이터 변환
-

문제 10: 데이터 분할 및 스케일링

문제: 훈련/검증 데이터로 분할하고 RobustScaler로 스케일링하세요.

답안:

python

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import RobustScaler

# Feature와 Target 분리
X = df_preset.drop('Time_Driving', axis=1)
y = df_preset['Time_Driving']

# 훈련/검증 데이터 분할
X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# RobustScaler로 스케일링
scaler = RobustScaler()
X_train = scaler.fit_transform(X_train)
X_valid = scaler.transform(X_valid)
```

핵심 개념:

- `train_test_split()`: 데이터 분할
- `RobustScaler`: 이상치에 강건한 스케일링
- `fit_transform()` vs `transform()`: 훈련 데이터로 학습 후 적용

문제 11: 머신러닝 모델 학습

문제: 의사결정나무와 랜덤포레스트 모델을 만들고 학습하세요.

답안:

python

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
```

```
# 의사결정나무 모델
```

```
dt = DecisionTreeRegressor(
    max_depth=5,
    min_samples_split=3,
    random_state=120
)
dt.fit(X_train, y_train)
```

```
# 랜덤포레스트 모델
```

```
rf = RandomForestRegressor(
    max_depth=5,
    min_samples_split=3,
    random_state=120
)
rf.fit(X_train, y_train)
```

핵심 개념:

- 의사결정나무: 규칙 기반 모델
- 랜덤포레스트: 앙상블 모델
- 하이퍼파라미터 튜닝의 중요성

문제 12: 모델 성능 평가

문제: MAE로 두 모델의 성능을 비교하세요.

답안:

```
python
```

```
from sklearn.metrics import mean_absolute_error
```

```
# 예측
```

```
y_pred_dt = dt.predict(X_valid)
```

```
y_pred_rf = rf.predict(X_valid)
```

```
# MAE 계산
```

```
dt_mae = mean_absolute_error(y_valid, y_pred_dt)
```

```
rf_mae = mean_absolute_error(y_valid, y_pred_rf)
```

```
print(f"Decision Tree MAE: {dt_mae}")
```

```
print(f"Random Forest MAE: {rf_mae}")
```

```
# 성능 좋은 모델 선택
```

```
if dt_mae < rf_mae:
```

```
    답안12 = 'decisiontree'
```

```
else:
```

```
    답안12 = 'randomforest'
```

핵심 개념:

- MAE(Mean Absolute Error): 평균 절대 오차
- 모델 성능 비교 및 평가
- 일반적으로 랜덤포레스트가 더 좋은 성능을 보임

문제 13: 딥러닝 모델 구축

문제: TensorFlow로 딥러닝 모델을 만들고 학습하세요.

답안:

```
python
```



```

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

tf.random.set_seed(1)

# 모델 구성
model = Sequential([
    Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    Dropout(0.2),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(1) # 회귀이므로 출력층은 1개 뉴런
])

# 모델 컴파일
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mse']
)

# 모델 학습
history = model.fit(
    X_train, y_train,
    epochs=30,
    batch_size=16,
    validation_data=(X_valid, y_valid),
    verbose=1
)

```

핵심 개념:

- Sequential 모델: 층을 순차적으로 쌓는 모델
- Dropout: 과적합 방지
- MSE: 회귀 문제의 손실 함수
- validation_data: 검증 데이터로 성능 모니터링

문제 14: 학습 과정 시각화

문제: 학습 MSE와 검증 MSE를 그래프로 표시하세요.

답안:

python

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.plot(history.history['mse'], label='mse')
plt.plot(history.history['val_mse'], label='val_mse')
plt.title('Model MSE')
plt.xlabel('Epochs')
plt.ylabel('MSE')
plt.legend()
plt.show()
```

핵심 개념:

- `history`: 학습 과정의 메트릭 저장
- 과적합 탐지: 검증 손실이 증가하기 시작하는 지점
- 시각화를 통한 모델 성능 분석

핵심 학습 포인트

1. 데이터 전처리 프로세스

1. 데이터 로딩 → 2. 결측치 처리 → 3. 이상치 제거 → 4. 인코딩 → 5. 스케일링

2. 모델링 접근법

- 전통적 ML: scikit-learn (의사결정나무, 랜덤포레스트)
- 딥러닝: TensorFlow/Keras

3. 성능 평가

- 회귀: MAE, MSE, RMSE
- 시각화: 학습 곡선 분석

4. 실무 팁

- 항상 데이터 탐색(EDA)부터 시작
- 검증 데이터로 성능 확인
- 과적합 방지를 위한 정규화 기법 활용
- 하이퍼파라미터 튜닝으로 성능 최적화

이 가이드를 통해 AICE 시험의 전체적인 플로우를 이해하고 실습할 수 있습니다. 각 단계별로 코드를 실행해보며 결과를 확인해보세요!

